

colorbar.py

```
1 from collections.abc import Callable
2 from tkinter import colorchooser
3 from PIL import Image, ImageTk
4 import tkinter as tk
5
6 class ColorBar(tk.Frame):
7     def __init__(self, master, on_color_change: Callable[[tuple], None], *args,
8         **kwargs):
9         super().__init__(master = master, *args, **kwargs)
10
11         self.callback = on_color_change
12
13         self.configure(
14             background = "lightgray",
15             height = 64,
16             relief = "raised",
17             border = 2,
18         )
19
20         self.pack_propagate(False)
21
22         # Primary + Secondary Colors
23
24         self.wrapper = tk.Frame(
25             master = self,
26             width = 36,
27             height = 36,
28             background = "#DFDFDF",
29             relief = "sunken",
30             border = 2
31         )
32
33         self.primaryColor = "black"
34         self.secondaryColor = "white"
35
36         self.primaryColorFrame = tk.Frame(
37             master = self.wrapper,
38             width = 12,
39             height = 12,
40             background = self.primaryColor,
41             relief = "sunken",
42             border = 1
43         )
44
45         self.secondaryColorFrame = tk.Frame(
46             master = self.wrapper,
```

```

47         height = 12,
48         background = self.secondaryColor,
49         relief = "sunken",
50         border = 1
51     )
52
53     self.secondaryColorFrame.place(relx = 0.6, rely = 0.6, anchor = "center")
54     self.primaryColorFrame.place(relx = 0.4, rely = 0.4, anchor = "center")
55     self.primaryColorFrame.lift()
56
57     self.wrapper.pack(side = "left", padx = 10)
58
59     # Bind to wrapper and children for proper hover and click event
60     for widget in self.wrapper.winfo_children() + [self.wrapper]:
61         widget.bind("<Enter>", lambda _: self.wrapper.configure(background =
"lightgray"))
62         widget.bind("<Leave>", lambda _: self.wrapper.configure(background =
"#DFDFDF"))
63         widget.bind("<Button-1>", lambda _: self.swapPrimarySecondary())
64
65     # Custom color gradient button
66     self.customColor = tk.Canvas(
67         self,
68         width = 32,
69         height = 32,
70         border = 2,
71         relief = "sunken",
72     )
73
74     # Resize with PIL
75     self.gradientButtonImage = Image.open("assets/gradient.png")
76     self.gradientButtonImage = self.gradientButtonImage.resize((48, 48))
77     self.gradientButtonImage = ImageTk.PhotoImage(self.gradientButtonImage)
78
79     self.customColor.create_image(0, 0, anchor = "nw", image =
self.gradientButtonImage)
80
81     self.customColor.pack(
82         anchor = "w",
83         padx = 10,
84         expand = True,
85         side = "right",
86     )
87
88     self.customColor.bind(
89         "<Button-1>",
90         lambda _: self.setPrimaryColor(colorchooser.askcolor()[1])
91     )
92

```

```

93         # Presets
94
95         self.presets = {
96             "columns": 10 ,
97             "rows": 2,
98             "size": 16,
99             "gap": 4,
100            "colors": [
101                ["red", "orange", "yellow", "green", "blue", "purple", "pink",
102                "brown", "black", "gray"],
103                ["darkred", "darkorange", "gold", "darkgreen", "darkblue",
104                "purple2", "deeppink2", "white", "white", "white"]
105            ]
106        }
107
108        for row in self.presets["colors"]:
109            while len(row) < self.presets["columns"]:
110                row.append("")
111
112        self.presetFrame = tk.Frame(
113            master = self,
114            width = self.presets["columns"] * (self.presets["size"] +
115            self.presets["gap"]),
116            height = self.presets["rows"] * (self.presets["size"] +
117            self.presets["gap"]),
118            relief = "sunken",
119            background = "lightgray"
120        )
121
122        self.presetFrame.pack(anchor = "w", side = "right")
123
124        for x in range(self.presets["columns"]):
125            for y in range(self.presets["rows"]):
126                frame = tk.Frame(
127                    master = self.presetFrame,
128                    width = self.presets["size"],
129                    height = self.presets["size"],
130                    relief = "sunken",
131                    border = 2,
132                    background = self.presets["colors"][y][x] or "white",
133                )
134
135                frame.grid(
136                    column = x,
137                    row = y,
138                    padx = self.presets["gap"] // 2,
139                    pady = self.presets["gap"] // 2
140                )
141
142        # Note: lambda for freezing

```

```

139         frame.bind(
140             "<Button-1>",
141             lambda _, x = x, y = y:
142                 self.setPrimaryColor(self.presets["colors"][y][x])
143         )
144
145         frame.bind(
146             "<Shift-Button-1>",
147             lambda _, frame = frame, x = x, y = y:
148                 (
149                     frame.configure(background = self.primaryColor), # Button
bg
150                     self.setPresetColor((x, y))
151                 )
152         )
153
154     def setPrimaryColor(self, color):
155         self.primaryColor = color
156         self.primaryColorFrame.configure(bg = color)
157         self.callback(self.getColor())
158
159     def setSecondaryColor(self, color):
160         self.secondaryColor = color
161         self.secondaryColorFrame.configure(bg = color)
162
163     def setPresetColor(self, location: tuple[int, int]):
164         self.presets["colors"][location[1]][location[0]] = self.primaryColor
165
166     def swapPrimarySecondary(self):
167         primary = self.primaryColor
168         secondary = self.secondaryColor
169
170         self.setPrimaryColor(secondary)
171         self.setSecondaryColor(primary)
172
173     def getColor(self) -> tuple[int, int, int]:
174         # Raw RGB 0..65535 to 0..255
175         # Type checking ignored because i know the value is correct
176         return tuple([color // 256 for color in
self.master.winfo_rgb(self.primaryColor)]) # type: ignore
177

```